MerchProo

Merchandise Production Management System with POS and Workflow Tracking

Member 1: Michael Andrei N. Baluyot
Member 2: John Vladimir L. Mones
Member 3: De Castro, Marwen
Member 4: Froa, Jaymar
Member 5: Valenzona, John Ivan

Overview

The project is designed to simulate an enterprise-level development environment. It is not merely a coding exercise but a rigorous demonstration of software engineering principles, requiring students to bridge the gap between high-level architectural design and low-level technical implementation. The final output must be a robust, persistent, and secure utility capable of managing complex data relationships and delivering actionable insights for social or environmental impact.

I. Introduction

- **1.1 Project Overview:** Vision and chosen UN SDG Target.

The Merchandise Production Management system with POS and workflow Tracking is designed to assist small merchandise production businesses such as custom printing shops or apparel production businesses in organizing their operations

The system allows users to record customer orders, assign production tasks, monitor workflow progress, and process payments through an integrated POS module. By combining these functions into a single system, business can maintain better record and track the progress of their operations

Vision

The vision of this project is to create a simple and organized digital system that helps merchandise production production businesses manage orders, production tasks, and sales transactions more efficiently

SDG 8 decent work and economic grow

SDG 9 industry, innovation and infrastructure
Addresses SDG objective; provides 3+ complete logical business processes and persistence.

The system helps improve productivity and organization in small business by providing tools for better order management and workflow monitoring**.** The system encourages the use of digital tools to improve operations processes and support innovation in small production businesses

- **1.2 Problem Statement:** Rationale for the application's existence.

Many small merchandise production businesses still manage their operations manually using notebooks, spreadsheets, or separate tools for orders, production tasks, and payments. This can lead to several problems such as:

- Difficulty tracking the status of customer orders
- Lack of proper task assignment for production staff
- Errors in recording sales transactions
- Poor organization of records
- Limited ability to monitor production progress and sales performance

Because of these issues, employees and managers may find it difficult to monitor the workflow of orders and manage business data effectively.

To address these problems, this project proposes a **Merchandise Production Management System with POS and Workflow Tracking** that integrates order management, production monitoring, and sales transactions into a single platform.

II. Requirements & Analysis

- **2.1 Functional Requirements:** (FR1, FR2, etc.)

FR1 – User Login
The system shall allow users to log in using a username and password.

FR2 – Customer Management
The system shall allow users to add, update, view, and delete customer records.

FR3 – Order Management
The system shall allow users to create and manage merchandise production orders.

FR4 – Workflow Tracking
The system shall allow employees to update the status of production tasks.

FR5 – POS Transaction Processing
The system shall allow cashiers to record payments for completed orders.

FR6 – Sales Recording
The system shall store transaction records for reporting purposes.

FR7 – Reporting
The system shall generate simple reports such as total sales or number of completed orders.

FR8 – Data Storage
The system shall store all data in a relational database.

- **2.2 Non-Functional Requirements:** Performance, Security, and Robustness metrics.

Performance

The system should process database operations and transactions within **2–3 seconds**.

Security

User passwords must be stored securely using **password hashing**.

Robustness

The system should implement **error handling mechanisms** to prevent application crashes.

Usability

The system interface should be simple and easy for users to understand.

Maintainability

The system should follow **N-Tier Architecture** to separate user interface, business logic, and database operations.

- **2.3 Complexity Analysis:** Big O Notation analysis for the core Business Logic method (e.g., the algorithm in the Service Layer).

One of the main processes in the system is updating the **workflow status of production tasks**.

Algorithm Steps

1. Retrieve all tasks related to an order.
2. Check the status of each task.
3. Update the order progress if all tasks are completed.
4. Save the updated data to the database.

Big-O Complexity

If the system checks **n tasks**, the algorithm must go through each task once.

Time Complexity:

O(n)

This means the processing time increases depending on the number of tasks being checked.

III. Design Specification

- **3.1 N-Tier Architecture Diagram:** Visual representation of the layer interactions.

Presentation Layer

(User Interface / Forms)

↓

Business Logic Layer

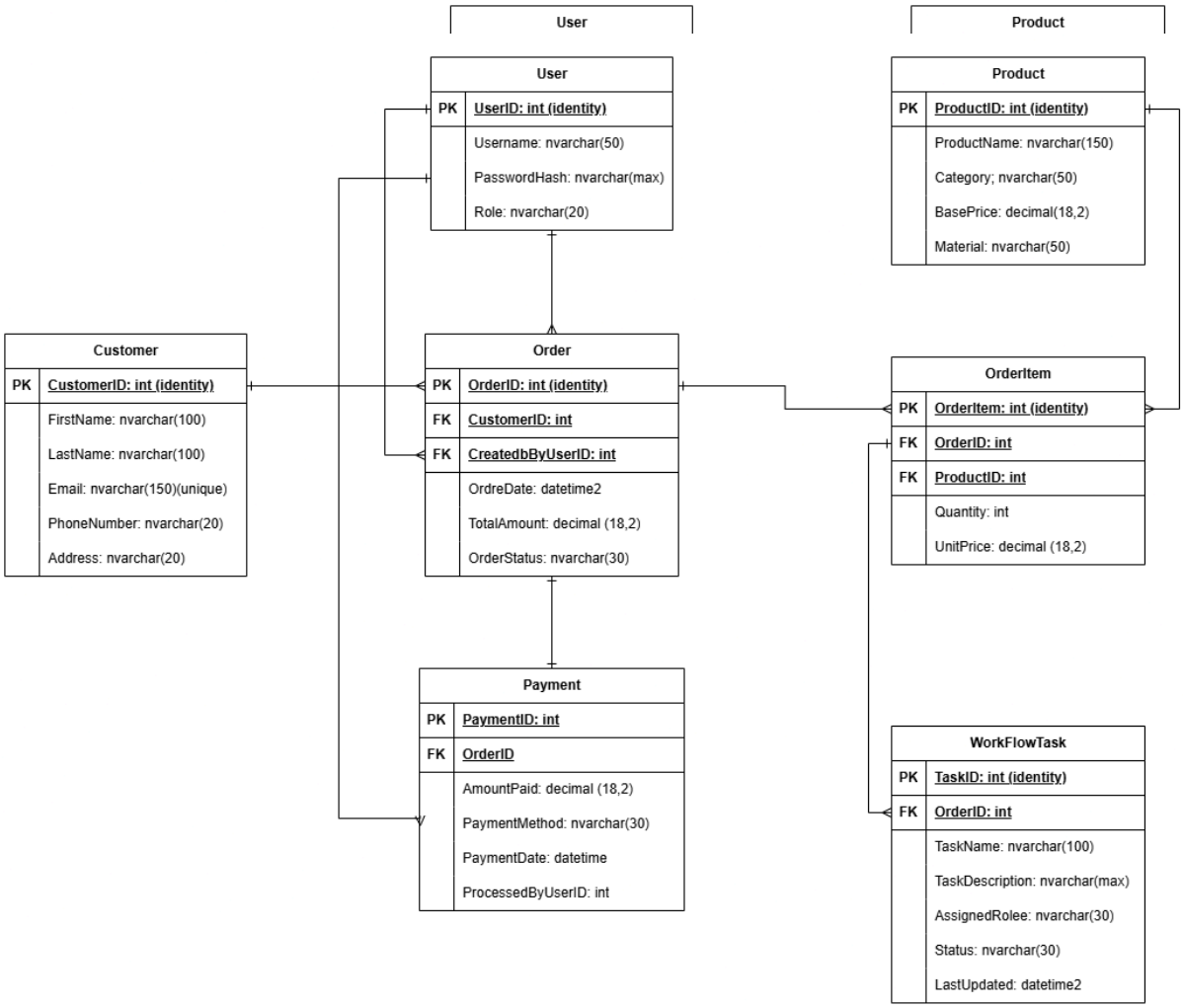
(System rules and processing)

↓

Data Access Layer

(Database operations)

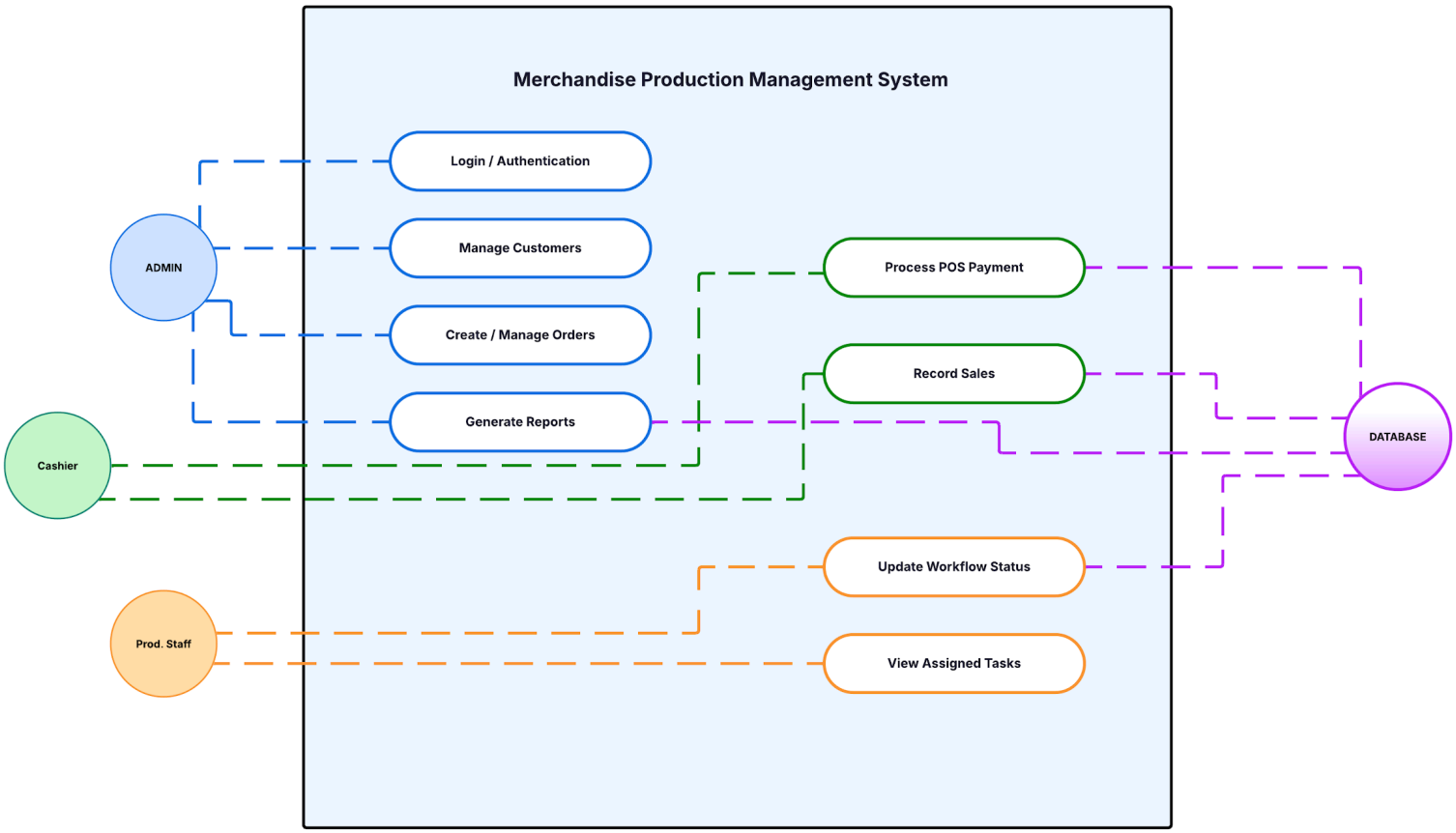
- **3.2 Entity Relationship Diagram (ERD):** Detailed database schema including data



types and Foreign Key relationships.

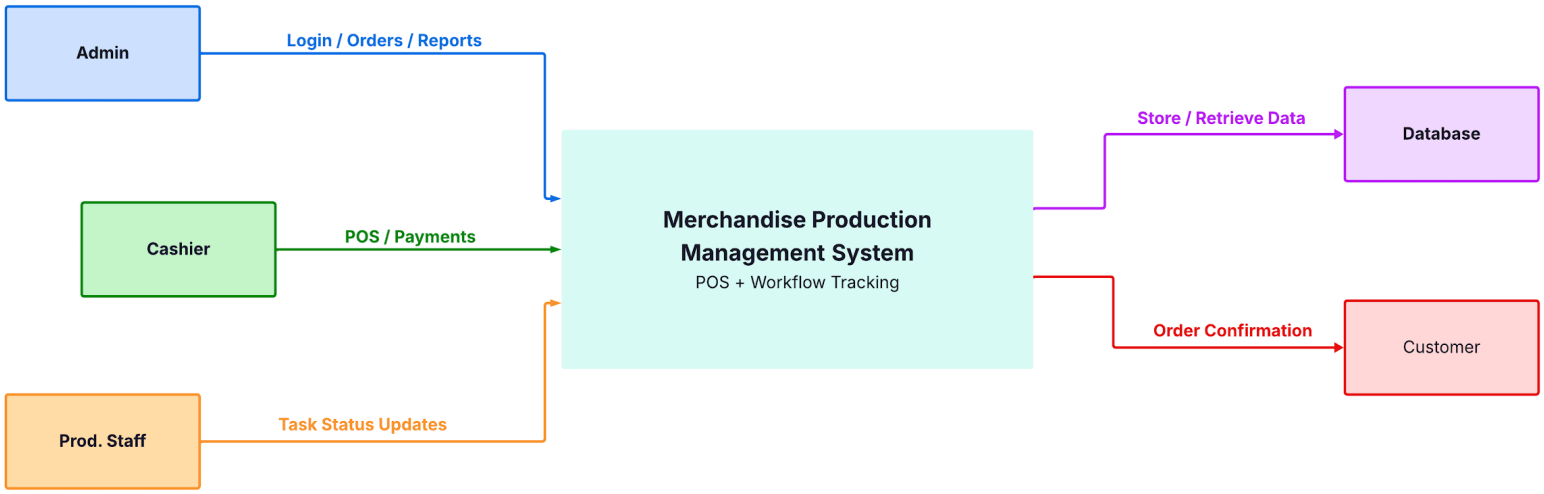
- **3.3 Use Case & Data Flow Diagrams (DFD):** Modeling user interactions and data movement.

Use Case Diagram



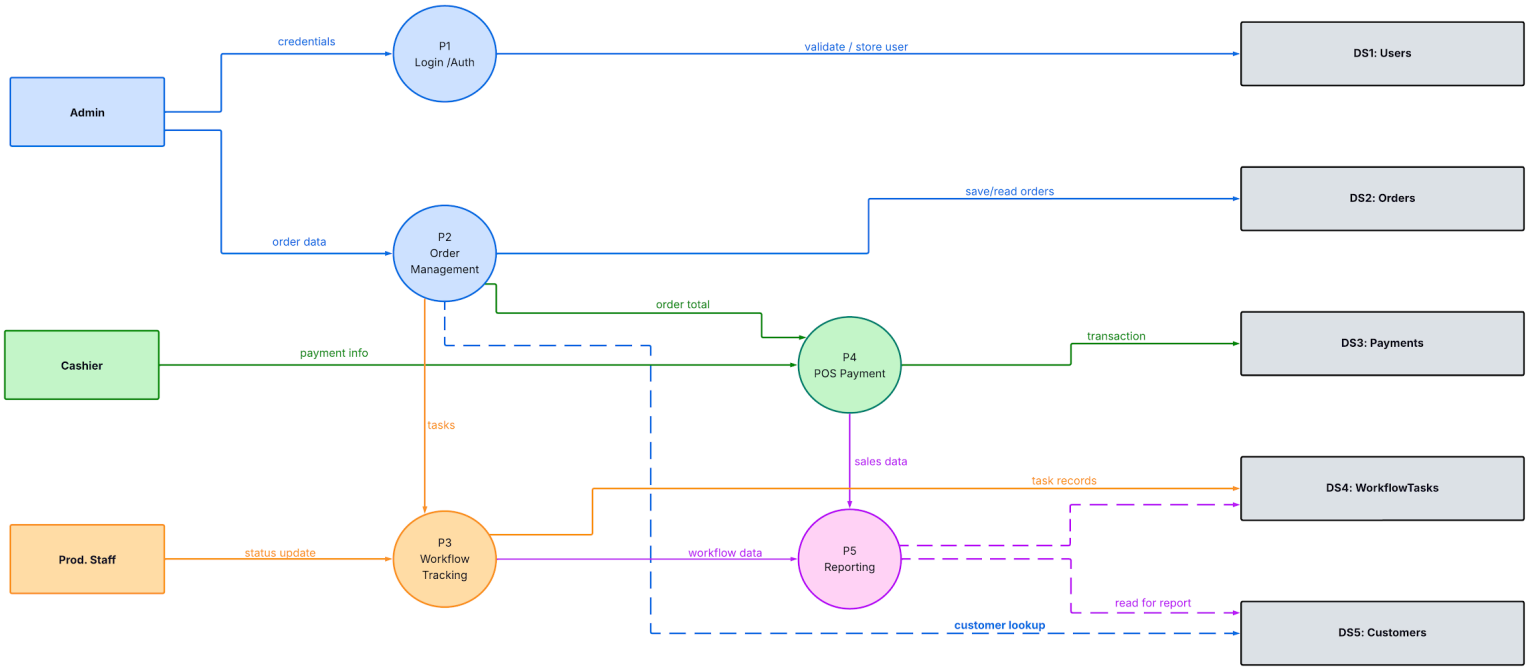
shows all three actors (Admin, Cashier, Production Staff) and their interactions with each functional module (Login, Order Management, POS, Workflow Tracking, Reports).

DFD Level 0



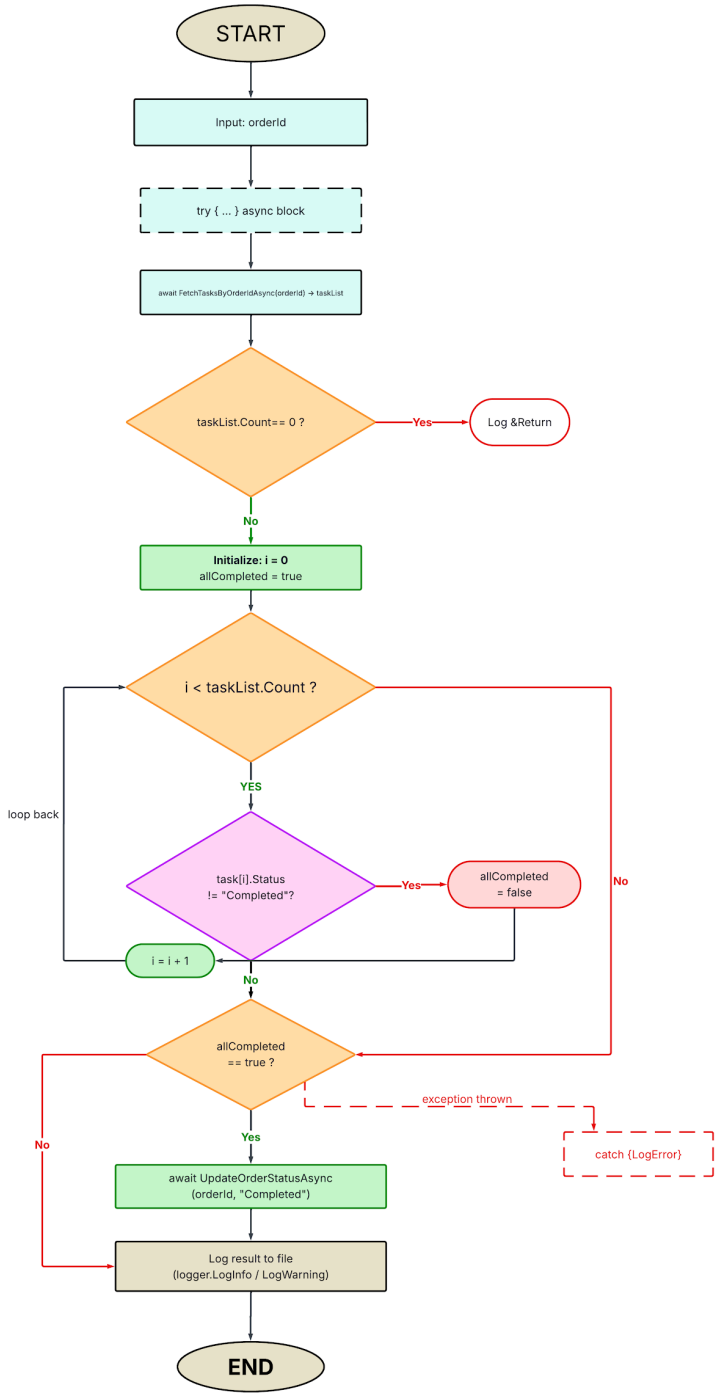
high-level view of the entire system as a single process, with all external entities (Admin, Cashier, Prod. Staff, Customer, Database) and their data flows in/out.

DFD Level 1



breaks the system into 5 internal processes (P1–P5) with labeled data flows between actors, processes, and 5 data stores (Users, Orders, Payments, WorkflowTasks, Customers).

- 3.4 Algorithm Flowchart: Step-by-step logic of the Async or Business method.



step by step async logic for UpdateWorkflowStatusAsync(), showing the O(n) loop through tasks, the allCompleted flag, the order status update, try-catch error handling, and logging all matching your Business Logic Layer requirements.

IV. Individual Contributions

- A table detailing exactly which Layer or Module was implemented by which member.

V. Appendices

- Screenshots and description of the application

Yung coding shit sa sabado na natin pag usapan

Mandatory Technical Requirements

To pass, the application must implement the following core concepts from the 14-week curriculum:

Category	Requirement
Architecture	Implementation of **N-Tier Architecture** (Presentation, Business Logic, and Data Access Layers).
OOP Pillars	Advanced use of **Inheritance**, **Interfaces**, and **Polymorphism**.
Persistence	**Database** integration via Entity Framework Core (EF Core) with at least two related tables.
Concurrency	Implementation of **Async/Await** for long-running tasks (e.g., Cloud Sync simulation).
Robustness	**Global Exception Handling** and structured **Logging** to an external file.
Data Exchange	**JSON Serialization** for backup/restore or reporting features.

2. UN SDG Alignment

Groups must select one SDG and build a utility around it. Examples include:

- **SDG 3 (Good Health):** Patient Records or Vaccine Distribution Tracker.
- **SDG 12 (Responsible Consumption):** Smart Inventory/Waste Management System.
- **SDG 13 (Climate Action):** Carbon Footprint Calculator or Energy Audit Tool.

3. Technical Specifications

Functional Requirements (FR)

- **FR1 (Data Management):** The system must perform full **CRUD** operations on a relational database.
- **FR2 (Business Logic):** The system must execute complex calculations (e.g., carbon credits, tax rates, or risk scores) inside a dedicated Service Layer.
- **FR3 (Visualization):** The system must display at least one **Graphical Chart** summarizing database data (e.g., Trends over time).
- **FR4 (Security):** The system must implement a **Hashed Login** system.

Non-Functional Requirements (NFR)

- **NFR1 (UI Responsiveness):** Heavy operations must not freeze the UI; use `async/await`.
- **NFR2 (Maintainability):** Code must follow N-Tier patterns. Forms should not contain SQL or math logic.
- **NFR3 (Reliability):** Every database and file operation must be wrapped in `try-catch` blocks with error logging.

References AI and https://sdgs.un.org/goals